

Two Sum

Three approaches, visualized execution, and the core logic to remember

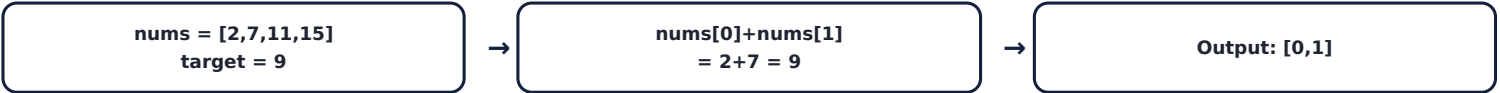
Brute Force · $O(n^2)$

HashMap · $O(n)$

Two Pointers · $O(n \log n)$

Problem Statement

Given an array `nums` and an integer `target`, return the indices of the two numbers that add up to `target`. Exactly one valid answer exists. You cannot reuse the same element twice. Order of the returned indices does not matter.



Approach Comparison

Approach	Time	Space	Returns	Core Idea
Brute Force	$O(n^2)$	$O(1)$	indices	Check every pair
HashMap	$O(n)$	$O(n)$	indices	Remember complements while scanning
Two Pointers	$O(n \log n)$	$O(1)^*$	values (not index)	Sort, then squeeze from both ends

Big picture: All three ask the same question differently — "does a partner for this number exist?" Brute force asks it n times per element, HashMap asks it once per element using memory, and two pointers exploits sorted order to answer it in a converging scan.

1 · Brute Force

O(n²) time

O(1) space

Pick i



Pick j



nums[i]+nums[j]
== target?



Yes: return [i,j]
No: try next pair

```
// for every element, compare against every other element
for i in 0..n:
  for j in 0..n:
    if nums[i] + nums[j] == target: return [i, j]
```

Trace for nums = [11,7,2,15], target = 9:

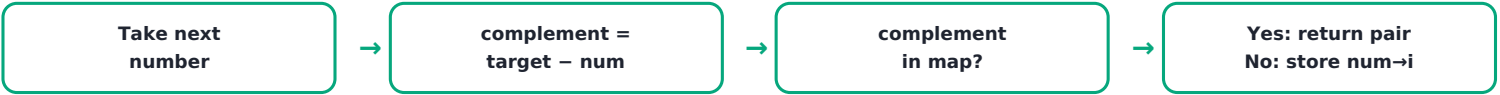
i	j	nums[i]	nums[j]	sum	match?
0	1	11	7	18	no
0	2	11	2	13	no
0	3	11	15	26	no
1	2	7	2	9	✓ yes

Recall trick: "Nested loop, check every pair." No memory of what's already been seen — that's exactly what the next approach fixes.

2 · HashMap (One-Pass)

O(n) time

O(n) space



```
// remember every number's index; ask "have I already seen my partner?"
for i in 0..n:
    complement = target - nums[i]
    if complement in map:
        return [map[complement], i]
    map[nums[i]] = i
```

Trace for nums = [11,7,2,15], target = 9:

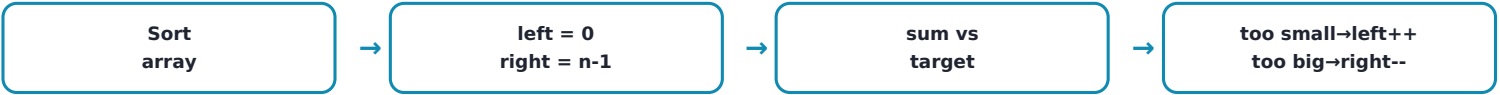
i	nums[i]	complement	in map?	map after step
0	11	-2	no	{11:0}
1	7	2	no	{11:0, 7:1}
2	2	7	✓ yes → index 1	return [1, 2]

Recall trick: "Have I already seen the number that completes me?" The map stores value → index for everything seen so far, turning an O(n) inner search into an O(1) lookup. This is the single insight that drops the complexity from O(n²) to O(n).

3 · Two Pointers (Sorted Array)

O(n log n) time

O(1) extra space



```
// sorted array lets us squeeze inward toward the target
sort(nums)
left = 0, right = n - 1
while left < right:
    sum = nums[left] + nums[right]
    if sum == target: return [nums[left], nums[right]]
    elif sum < target: left++
    else: right--
```

Trace for sorted nums = [2,7,11,15], target = 9:

left	right	nums[left]	nums[right]	sum	action
0	3	2	15	17	too big → right--
0	2	2	11	13	too big → right--
0	1	2	7	9	✓ match → return [2,7]

Important gotcha: sorting destroys the original indices. This approach naturally returns **values**, not indices — a different contract than the problem asks for. Only use it directly when values are acceptable, or keep an index map built *before* sorting.

Recall trick: "Sorted array, squeeze from both ends." Sum too small → move left pointer right (increase sum). Sum too big → move right pointer left (decrease sum). Pointers converge until they meet or cross.

What to remember

- **Default choice:** HashMap, one pass, $O(n)$ — "store what I've seen, check if its complement already exists."
- **Brute force** is the fallback when nothing else comes to mind, but it's $O(n^2)$ — fine for small inputs or as a starting point in an interview.
- **Two pointers** is elegant and $O(n \log n)$, but only works cleanly when original indices don't matter, since sorting reorders everything.
- **Underlying pattern across all three:** "does a number's partner exist?" — Reference notes: TwoSum.java brute force checks exhaustively, HashMap remembers, two pointers exploits order.